

```
class Rectangle: public Shape {
public:
    virtual void draw(ShapeColor color = Red) const;
    ...
};
```

喔欧, 代码重复。更糟的是, 代码重复又带着相依性 (with dependencies): 如果 Shape 内的缺省参数值改变了, 所有“重复给定缺省参数值”的那些 derived classes 也必须改变, 否则它们最终会导致“重复定义一个继承而来的缺省参数值”。怎么办?

当你想令 virtual 函数表现出你所想要的行为但却遭遇麻烦, 聪明的做法是考虑替代设计。条款 35 列了不少 virtual 函数的替代设计, 其中之一是 NVI (*non-virtual interface*) 手法: 令 base class 内的一个 public non-virtual 函数调用 private virtual 函数, 后者可被 derived classes 重新定义。这里我们可以让 non-virtual 函数指定缺省参数, 而 private virtual 函数负责真正的工作:

```
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };
    void draw(ShapeColor color = Red) const           //如今它是 non-virtual
    {
        doDraw(color);                               //调用一个 virtual
    }
    ...
private:
    virtual void doDraw(ShapeColor color) const = 0; //真正的工作
};                                                    //在此处完成

class Rectangle: public Shape {
public:
    ...
private:
    virtual void doDraw(ShapeColor color) const;     //注意, 不须指定
    ...                                              //缺省参数值。
};
```

由于 non-virtual 函数应该绝对不被 derived classes 覆写 (见条款 36), 这个设计很清楚地使得 draw 函数的 color 缺省参数值总是为 Red。

请记住

- 绝对不要重新定义一个继承而来的缺省参数值, 因为缺省参数值都是静态绑定, 而 virtual 函数——你唯一应该覆写的东西——却是动态绑定。

条款 38：通过复合塑模出 **has-a** 或“根据某物实现出”

Model "has-a" or "is-implemented-in-terms-of" through composition.

复合（composition）是类型之间的一种关系，当某种类型的对象内含它种类型的对象，便是这种关系。例如：

```
class Address { ... };           //某人的住址
class PhoneNumber { ... };

class Person {
public:
    ...
private:
    std::string name;           //合成成分物（composed object）
    Address address;           //同上
    PhoneNumber voiceNumber;   //同上
    PhoneNumber faxNumber;     //同上
};
```

本例之中 Person 对象由 string, Address, PhoneNumber 构成。在程序员之间复合（composition）这个术语有许多同义词，包括 *layering*（分层），*containment*（内含），*aggregation*（聚合）和 *embedding*（内嵌）。

条款 32 曾说，“public 继承”带有 **is-a**（是一种）的意义。复合也有它自己的意义。实际上它有两个意义。复合意味 **has-a**（有一个）或 **is-implemented-in-terms-of**（根据某物实现出）。那是因为你正打算在你的软件中处理两个不同的领域（domains）。程序中的对象其实相当于你所塑造的世界中的某些事物，例如人、汽车、一张张视频画面等等。这样的对象属于应用域（*application domain*）部分。其他对象则纯粹是实现细节上的人工制品，像是缓冲区（*buffers*）、互斥器（*mutexes*）、查找树（*search trees*）等等。这些对象相当于你的软件的实现域（*implementation domain*）。当复合发生于应用域内的对象之间，表现出 **has-a** 的关系；当它发生于实现域内则是表现 **is-implemented-in-terms-of** 的关系。

上述的 Person class 示范 **has-a** 关系。Person 有一个名称，一个地址，以及语音和传真两笔电话号码。你不会说“人是一个名称”或“人是一个地址”，你会说“人有一个名称”和“人有一个地址”。大多数人接受此一区别毫无困难，所以很少人会对 **is-a** 和 **has-a** 感到困惑。

比较麻烦的是区分 **is-a**（是一种）和 **is-implemented-in-terms-of**（根据某物实现出）这两种对象关系。假设你需要一个 **template**，希望制造出一组 **classes** 用来表现由不重复对象组成的 **sets**。由于复用（*reuse*）是件美妙无比的事情，你的第一个